poorly written bot. Or savvy users might hack your URLs and add the name of a city that does not exist in your database.

It may be savvy to prevent the wasted processing these database lookups for nonexistent values incur and inform the requester that the page they are looking for doesn't exist. In this case, you want to return a 404 Not Found status code.

The following steps show you how to create your own route constraint that checks the value for the `cityName` parameter against a predetermined collection of cities. If the route data item is not in the list, the match fails, and the application returns a 404 Not Found status code. The collection in this example is a simple array. In a real-world application, the data will reside in a database, but you would perform the lookup against a cached version of the cities to prevent the need to make a database call. In chapter 14, you will explore the use of the caching features offered by ASP.NET Core to query a collection stored in memory on the web server.

Route constraints are classes that implement the `IRouteConstraint` interface. So the first step is to create a C# class named `CityRouteConstraint`. This should be placed in a folder named RouteConstraints, so you don't have miscellaneous code files cluttering up the root folder of your project. The code for the `CityRouteConstraint` class is detailed in the following listing.

**Listing 4.5 The custom `CityRouteConstraint` class**

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Routing;
using System;
using System.Linq;

namespace CityBreaks.RouteContraints
{
    public class CityRouteConstraint : IRouteConstraint                    ❶
```